

Atelier 12 : **Javascript**

SOMMAIRE

Javascript 2/2 : Navigateur 1.....	3
Exercice.....	3
Enfants DOM.....	3
La question des frères et soeurs.....	5
Sélectionner toutes les cellules diagonales.....	6
Recherche d'éléments.....	7
Trouver le tableau avec id="age-table" :.....	7
Trouver tous les éléments <label> dans ce tableau :.....	7
Trouver le premier <td> dans ce tableau (avec le mot "Age") :.....	7
Trouver le premier <input> dans ce formulaire :.....	7
Trouver le dernier <input> dans ce formulaire :.....	7
Compter les descendants.....	10
Balise dans le commentaire.....	11
Où est le "document" dans la hiérarchie ?.....	12
Où est le document dans la hiérarchie ?.....	12
À quelle classe appartient le document ?.....	12
Quelle est sa place dans la hiérarchie DOM ?.....	12
Hérite-t-il de Node, Element ou HTMLInputElement ?.....	12
Javascript 2/2 : Navigateur 2.....	13
Obtenez l'attribut.....	13
Rendre les liens externes orange.....	15
createTextNode.....	17
elem.append(document.createTextNode(text)).....	17
elem.innerHTML = text.....	17
elem.textContent = text.....	17
Effacer l'élément.....	18
Pourquoi "aaa" reste-t-il ?.....	19
Créer une liste.....	20
Créer un arbre à partir de l'objet.....	22
Afficher les descendants dans un arbre.....	24
Créer un calendrier.....	26
Horloge colorée avec setInterval.....	28
Trier le tableau.....	29
Create a notification.....	30
Quel est le défilement.....	32
Quelle est la largeur de la barre de défilement ?.....	33

Placer la balle au centre du terrain.....	34
La différence : largeur CSS vs clientWidth.....	36
Trouver les coordonnées de la fenêtre du champ.....	37
Afficher une note près de l'élément.....	40

Javascript 2/2 : Navigateur 1

Exercice

Enfants DOM

Enfants DOM

Regardez cette page :

```
1 <html>
2 <body>
3   <div>Users:</div>
4   <ul>
5     <li>John</li>
6     <li>Pete</li>
7   </ul>
8 </body>
9 </html>
```

Pour chacun des éléments suivants, donnez au moins un moyen d'y accéder :

- Le noeud `<div>` du DOM ?
- Le noeud `<ul>` du DOM ?
- Le deuxième `<li>` (avec Pete) ?

Dans cet exercice, nous allons étudier la structure du DOM à partir d'un code HTML contenant une balise `<div>`, une liste `<ul>` et deux éléments `<li>`. Le but de cet exercice est d'apprendre à accéder aux éléments du DOM avec JavaScript

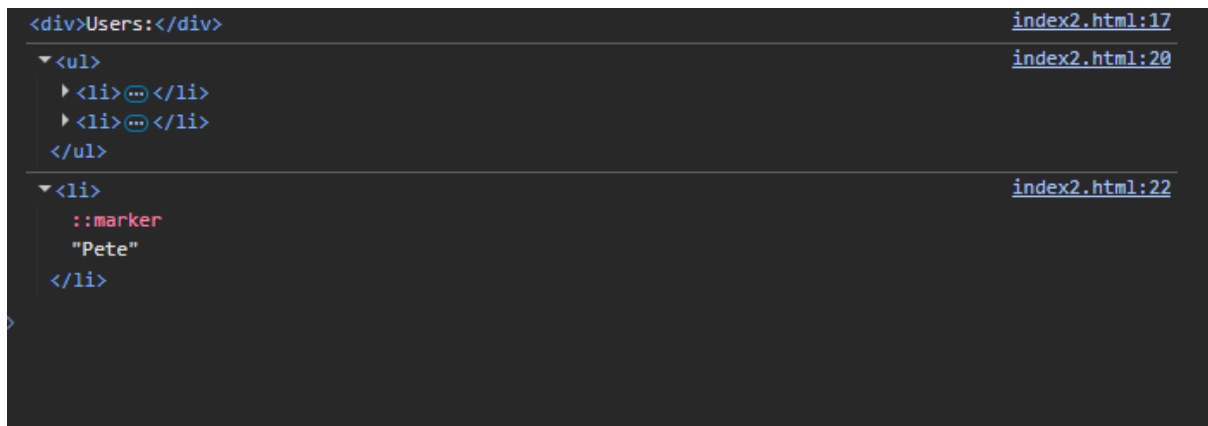
```
<script>

  console.log(document.querySelector("div"));

  console.log(document.querySelector("ul"));

  console.log(document.querySelectorAll("li")[1]);
</script>
```

Nous allons maintenant ouvrir la page HTML dans le navigateur et appuyer sur la touche F12, le but est de vérifier que le code JavaScript permet bien d'accéder aux nœuds du DOM.



```
<div>Users:</div>                                     index2.html:17
└─<ul>                                                 index2.html:20
  │   <li>⋮ </li>
  │   <li>⋮ </li>
  └─</ul>
└─<li>                                                 index2.html:22
  │   ::marker
  │   "Pete"
  └─</li>
```

En ouvrant la console du navigateur, nous pouvons observer les résultats affichés par le script. Sur la capture d'écran, on voit que la console affiche le nœud `<div>` contenant le texte « Users: », le nœud `<ul>` et le deuxième nœud `<li>`, ce qui montre que l'accès aux éléments du DOM fonctionne correctement.

La question des frères et soeurs.

Si `element` – est un nœud élément arbitraire du DOM ...

- Est-il vrai que `elem.lastChild.nextSibling` est toujours `null` ?
- Est-il vrai que `elem.children[0].previousSibling` est toujours `null` ?

Est-il vrai que `elem.lastChild.nextSibling` est toujours `null` ? Oui, `lastChild` désigne le dernier enfant d'un nœud. Comme c'est le dernier, il n'a pas de frère suivant. Donc `nextSibling` renvoie toujours `null`.

Est-il vrai que `elem.children[0].previousSibling` est toujours `null` ? Non, ce n'est pas toujours vrai, `children[0]` est le premier enfant élément, mais `previousSibling` peut être : un nœud texte ou un commentaire. Donc même si c'est le premier élément, il peut quand même avoir un `previousSibling`.

Sélectionner toutes les cellules diagonales

Sélectionner toutes les cellules diagonales

Écrivez le code pour colorer toutes les cellules du tableau diagonal en rouge.

Vous devrez obtenir toutes les diagonales `<td>` de la `<table>` et les colorer en utilisant le code :

```
1 // td doit être la référence à la cellule du tableau
2 td.style.backgroundColor = 'red';
```

Le résultat devrait être :

1:1	2:1	3:1	4:1	5:1
1:2	2:2	3:2	4:2	5:2
1:3	2:3	3:3	4:3	5:3
1:4	2:4	3:4	4:4	5:4
1:5	2:5	3:5	4:5	5:5

Dans cet exercice, nous allons utiliser JavaScript pour accéder au tableau présent dans la page et parcourir ses lignes afin de repérer les cellules situées sur la diagonale. Nous allons ensuite sélectionner chaque cellule diagonale et modifier son style pour changer sa couleur de fond en rouge, ce qui nous permettra de vérifier que nous savons manipuler les éléments du DOM.

```
let table = document.querySelector("table");

for (let i = 0; i < table.rows.length; i++) {
  let td = table.rows[i].cells[i];
  td.style.backgroundColor = "red";
}
```

Nous voyons le code JavaScript que nous avons écrit pour résoudre l'exercice. Nous commençons par sélectionner le tableau de la page à l'aide de `document.querySelector("table")`. Ensuite, nous parcourons toutes les lignes du tableau avec une boucle `for`. À chaque itération, nous récupérons la cellule située sur la diagonale en utilisant l'indice de la ligne et de la colonne, puis nous changeons sa couleur de fond en rouge.

Recherche d'éléments

Recherche d'éléments

Voici le document avec le tableau et formulaire

Comment trouver ?...

1. Le tableau avec `id="age-table"`.
2. Tous les éléments `label` dans ce tableau (il devrait y en avoir 3).
3. Le premier `td` dans ce tableau (avec le mot "Age").
4. Le `form` avec `name="search"`.
5. Le premier `input` dans ce formulaire.
6. Le dernier `input` dans ce formulaire.

Trouver le tableau avec `id="age-table"` :

```
document.getElementById("age-table");
```

Trouver tous les éléments `<label>` dans ce tableau :

```
document.querySelectorAll("#age-table label");
```

Trouver le premier `<td>` dans ce tableau (avec le mot "Age") :

```
document.querySelector("#age-table td");
```

Trouver le `<form>` avec `name="search"` :

```
document.querySelector('form[name="search"]');
```

Trouver le premier `<input>` dans ce formulaire :

```
document.querySelector('form[name="search"] input');
```

Trouver le dernier `<input>` dans ce formulaire :

```
let form = document.querySelector('form[name="search"]');  
form.querySelectorAll("input")[form.querySelectorAll("input").length - 1];
```


Ouvrez la page table.html dans un onglet à part et utilisez les outils du navigateur pour cela.

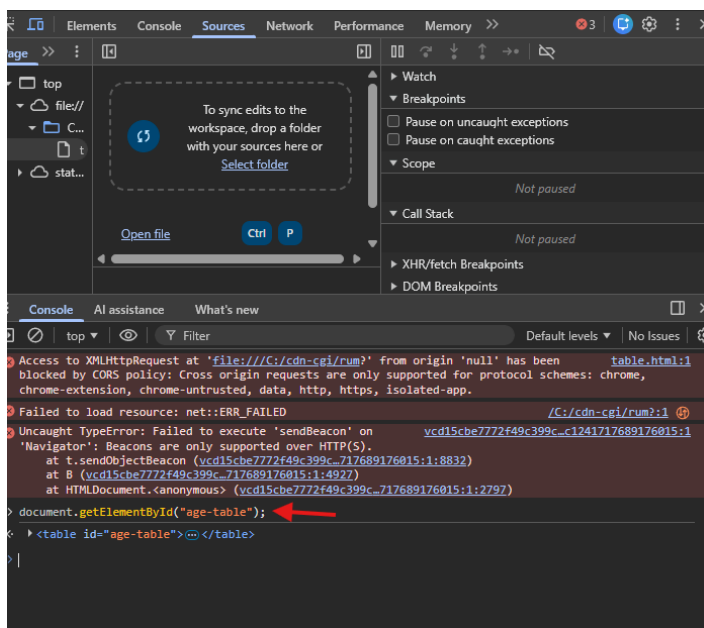
Cherchez dans le site :

Recherchez les visiteurs :

Age: ☐ moins de 18 ☐ 18-50 ☐ plus de 50

Infos complémentaires :

Nous allons ouvrir les outils de développement du navigateur afin d'accéder à la console JavaScript. Cela nous permet de tester les commandes du DOM et de vérifier que nous retrouvons correctement les éléments demandés dans la page, comme le tableau ou le formulaire.



Ensuite, nous avons tapé la commande "document.getElementById("age-table")" dans la console afin de rechercher le tableau à l'aide de son identifiant. La console a affiché l'élément <table id="age-table">, ce qui montre que nous avons bien retrouvé le tableau dans le DOM et que la commande fonctionne correctement.

Ensuite, nous avons répété la même démarche pour les autres commandes demandées. Nous avons utilisé la console afin de rechercher les différents éléments du document : les balises `<label>` présentes dans le tableau, le premier `<td>`, le formulaire ayant l'attribut `name="search"`, puis le premier et le dernier champ `<input>` de ce formulaire.

```
> document.getElementById("age-table");
< ▶ <table id="age-table">⋮ </table>
> document.querySelectorAll("#age-table label");
< ▶ NodeList(3) [Label, Label, Label]
> document.querySelector("#age-table td");document.querySelector('form[name="search"]');
< ▶ <form name="search">⋮ </form>
> document.querySelector('form[name="search"] input');
< <input type="text" name="search">
> let form = document.querySelector('form[name="search"]');
  form.querySelectorAll("input")[form.querySelectorAll("input").length - 1];
< <input type="submit" value="Search!">
>
```

Compter les descendants

Compter les descendants

Qu'affiche le script ?

```
1 <html>
2
3 <body>
4   <script>
5     alert(document.body.lastChild.nodeType);
6   </script>
7 </body>
8
9 </html>
```

```
<script>
|   alert(document.body.lastChild.nodeType);
</script>
```

Le script affiche 1 ou 3 selon la structure du document. Si le dernier nœud du body est un élément (comme la balise script), la valeur affichée est 1. S'il s'agit d'un nœud texte correspondant à un retour à la ligne ou à des espaces, la valeur affichée est 3.

Dans notre cas, le script affiche 1 car le dernier nœud du body est un nœud élément (la balise script) et non un nœud texte.

Cette page indique

1

OK

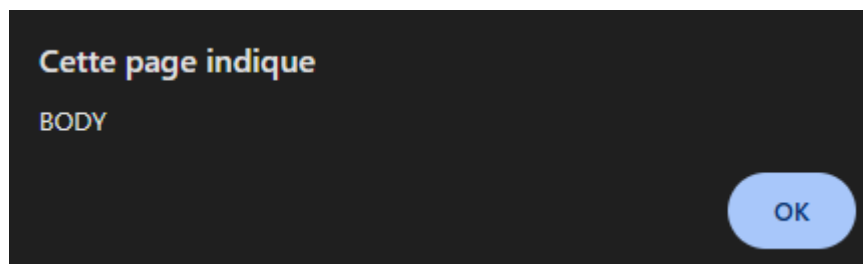
Balise dans le commentaire

Balise dans le commentaire

Qu'affiche ce code ?

```
1 <script>
2   let body = document.body;
3
4   body.innerHTML = "<!--" + body.tagName + "-->";
5
6   alert( body.firstChild.data ); // Qu'est ce qu'il y a ici ?
7 </script>
```

Le code affiche « BODY ».



Le code affiche « BODY », car le contenu du body est remplacé par un commentaire HTML et la propriété du nœud commentaire contient le texte du commentaire.

Où est le “document” dans la hiérarchie ?

Où est le "document" dans la hiérarchie ?

À quelle classe appartient le `document` ?

Quelle est sa place dans la hiérarchie DOM ?

Hérite-t-il de `Node` ou `Element`, ou peut-être de `HTMLElement` ?

Où est le document dans la hiérarchie ?

Le document se situe tout en haut de la hiérarchie du DOM. Il représente l'ensemble de la page HTML et constitue le point d'entrée pour accéder à tous les autres nœuds (html, body, éléments, etc.).

À quelle classe appartient le document ?

Le document appartient à la classe Document.

Quelle est sa place dans la hiérarchie DOM ?

Dans la hiérarchie du DOM, le document est le nœud racine.
Tous les autres nœuds de la page sont des descendants du document.

Hérite-t-il de Node, Element ou HTMLElement ?

Le document hérite de Node.

Le document est donc un nœud, mais pas un élément HTML.

Javascript 2/2 : Navigateur 2

Obtenez l'attribut

Obtenez l'attribut

Écrivez le code pour sélectionner l'élément avec l'attribut `data-widget-name` dans le document et pour lire sa valeur.

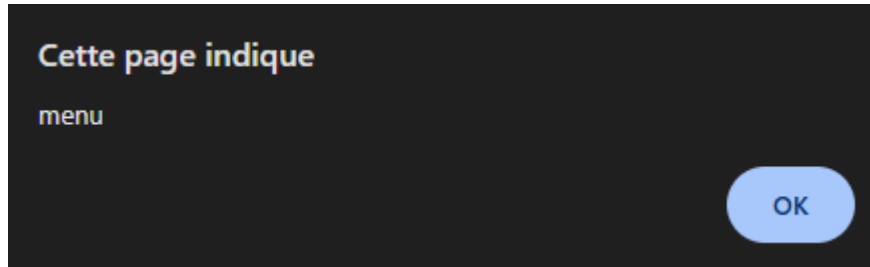
```
1 <!DOCTYPE html>
2 <html>
3 <body>
4
5   <div data-widget-name="menu">Choose the genre</div>
6
7   <script>
8     /* your code */
9   </script>
10 </body>
11 </html>
```

Dans cet exercice, nous allons sélectionner un élément du document à l'aide d'un attribut (`data-widget-name`) puis lire la valeur de cet attribut en utilisant Javascript.

Ensuite, nous allons écrire le code dans la balise `<script>`. Nous commençons par sélectionner l'élément qui possède l'attribut `data-widget-name` grâce à un sélecteur DOM. Une fois cet élément récupéré, nous allons lire la valeur de l'attribut à l'aide de la méthode `getAttribute`.

```
<script>
  let element = document.querySelector('[data-widget-name]');
  let value = element.getAttribute('data-widget-name');
  alert(value);
</script>
```

Nous allons vérifier que notre code fonctionne correctement en ouvrant la page HTML dans le navigateur. Lorsque la page se charge, le script s'exécute automatiquement et une fenêtre d'alerte s'affiche avec la valeur de l'attribut `data-widget-name`. Cette valeur est « menu », ce qui confirme que nous avons bien sélectionné le bon élément et que nous avons correctement lu la valeur de son attribut.



Rendre les liens externes orange

Rendre les liens externes orange

Mettez tous les liens externes en orange en modifiant leur propriété `style`.

Un lien est externe si :

- Son `href` contient `://`
- Mais ne commence pas par `http://internal.com`.

Exemple :

```
1 <a name="list">the list</a>
2 <ul>
3   <li><a href="http://google.com">http://google.com</a></li>
4   <li><a href="/tutorial">/tutorial.html</a></li>
5   <li><a href="local/path">local/path</a></li>
6   <li><a href="ftp://ftp.com/my.zip">ftp://ftp.com/my.zip</a></li>
7   <li><a href="http://nodejs.org">http://nodejs.org</a></li>
8   <li><a href="http://internal.com/test">http://internal.com/test</a></li>
9 </ul>
10
11 <script>
12   // setting style for a single link
13   let link = document.querySelector('a');
14   link.style.color = 'orange';
15 </script>
```

Le résultat devrait être :

The list:

- <http://google.com>
- </tutorial.html>
- <local/path>
- <ftp://ftp.com/my.zip>
- <http://nodejs.org>
- <http://internal.com/test>

Nous allons parcourir tous les liens de la page et identifier ceux qui sont des liens externes. Un lien est considéré comme externe si son attribut href contient `://` mais ne commence pas par `http://internal.com`. Une fois ces liens repérés, nous allons modifier leur style afin de les afficher en orange.

Nous commençons par sélectionner tous les liens <a> de la page. Ensuite, nous parcourons chaque lien à l'aide d'une boucle. Pour chaque lien, nous récupérons la valeur de l'attribut href et nous vérifions s'il contient :// afin d'identifier un lien externe. Nous vérifions aussi qu'il ne commence pas par http://internal.com. Si ces conditions sont respectées, nous modifions la propriété style.color du lien pour le mettre en orange. Le résultat permet de distinguer visuellement les liens externes des autres liens.

```
<script>
  let links = document.querySelectorAll("a");

  for (let link of links) {
    let href = link.getAttribute("href");

    if (href && href.includes("://") && !href.startsWith("http://internal.com")) {
      link.style.color = "orange";
    }
  }
</script>
```

Résultat

the list

- <http://google.com>
- </tutorial.html>
- <local/path>
- <ftp://ftp.com/my.zip>
- <http://nodejs.org>
- <http://internal.com/test>

createTextNode

createTextNode vs innerHTML vs textContent

Nous avons un élément DOM vide `elem` et une chaîne de caractères `text`.

Lesquelles de ces 3 commandes feront exactement la même chose ?

1. `elem.append(document.createTextNode(text))`
2. `elem.innerHTML = text`
3. `elem.textContent = text`

Nous allons comparer trois façons différentes d'ajouter du texte dans un élément du DOM afin de comprendre lesquelles produisent exactement le même résultat.

`elem.append(document.createTextNode(text))`

Cette commande crée un nœud texte et l'ajoute dans l'élément.
Le texte est inséré tel quel, sans être interprété comme du HTML.

`elem.innerHTML = text`

Cette commande insère le contenu en l'interprétant comme du HTML.
Si la variable `text` contient des balises HTML, elles seront analysées et transformées en éléments HTML.

`elem.textContent = text`

Cette commande insère uniquement du texte brut.
Les balises éventuelles sont affichées comme du texte et ne sont pas interprétées.

Les commandes `elem.append(document.createTextNode(text))` et `elem.textContent = text` font exactement la même chose. `elem.innerHTML = text` est différente car elle interprète le contenu comme du HTML.

Effacer l'élément

Effacer l'élément

Créez une fonction `clear(elem)` qui supprime tout de l'élément.

```
1 <ol id="elem">
2   <li>Hello</li>
3   <li>World</li>
4 </ol>
5
6 <script>
7   function clear(elem) { /* votre code */ }
8
9   clear(elem); // efface la liste
10 </script>
```

A présent nous allons créer une fonction JavaScript `clear(elem)` dont le but est de supprimer tout le contenu d'un élément du DOM. Nous allons ensuite appeler cette fonction pour effacer complètement la liste affichée dans la page.

```
<script>
  function clear(elem) {
    elem.innerHTML = "";
  }

  let elem = document.getElementById("elem");
  clear(elem);
</script>
```

Nous commençons par définir une fonction `clear(elem)` qui prend un élément du DOM en paramètre. À l'intérieur de cette fonction, nous vidons le contenu de l'élément en utilisant la propriété `innerHTML`. Ensuite, nous sélectionnons la liste grâce à son identifiant et nous appelons la fonction `clear` pour effacer tous ses éléments enfants. Le résultat est que la liste est totalement supprimée de l'affichage.

Pourquoi "aaa" reste-t-il ?

Pourquoi "aaa" reste-t-il ?

Dans l'exemple ci-dessous, l'appel `table.remove()` supprime le tableau du document.

mais si vous l'exécutez, vous pouvez voir que le texte "aaa" est toujours visible.

Pourquoi cela se produit-il ?

```
1 <table id="table">
2   aaa
3   <tr>
4     <td>Test</td>
5   </tr>
6 </table>
7
8 <script>
9   alert(table); // la table, comme il se doit
10
11   table.remove();
12   // pourquoi y a-t-il encore "aaa" dans le document ?
13 </script>
```

Nous allons supprimer un tableau du document avec JavaScript et comprendre pourquoi un texte placé à l'intérieur de la balise `<table>` reste visible après la suppression du tableau.

```
<script>
  let table = document.getElementById("table");
  alert(table);

  table.remove();
</script>
```

aaa

Le texte `aaa` reste visible car il n'est pas réellement contenu dans le tableau dans le DOM. Le navigateur déplace automatiquement ce texte en dehors de la balise `<table>`. Lorsque le tableau est supprimé, le texte reste donc affiché.

Créer une liste

Créer une liste

Écrivez une interface pour créer une liste à partir des entrées utilisateur.

Pour chaque élément de la liste :

1. Interrogez un utilisateur sur son contenu en utilisant `prompt`.
2. Créez le `<li>` avec et ajoutez-le à `<ul>`.
3. Continuez jusqu'à ce que l'utilisateur annule l'entrée (en appuyant sur la touche `Esc` ou une entrée vide).

Tous les éléments doivent être créés dynamiquement.

Si un utilisateur tape des balises HTML, elles doivent être traitées comme un texte.

```
<script>
  let ul = document.getElementById("list");

  while (true) {
    let text = prompt("Entrez un élément de la liste :");

    if (text === null || text === "") {
      break;
    }

    let li = document.createElement("li");
    li.textContent = text;
    ul.append(li);
  }
</script>
```

Nous commençons par sélectionner la liste `<ul>` dans le document. Ensuite, nous utilisons une boucle qui affiche une fenêtre `prompt` pour demander un texte à l'utilisateur. Tant que l'utilisateur entre un texte valide, nous créons un nouvel élément `<li>` et nous y plaçons le contenu avec `textContent`.

Cette page indique

Entrez un élément de la liste :

OK Annuler

- bonjour

Créer un arbre à partir de l'objet

Créer un arbre à partir de l'objet

Écrivez une fonction `createTree` qui crée une liste imbriquée `ul/li` à partir de l'objet imbriqué.

Par exemple :

```

1 let data = {
2   "Fish": {
3     "trout": {},
4     "salmon": {}
5   },
6
7   "Tree": {
8     "Huge": {
9       "sequoia": {},
10      "oak": {}
11    },
12    "Flowering": {
13      "apple tree": {},
14      "magnolia": {}
15    }
16  }
17 };

```

La syntaxe :

```

1 let container = document.getElementById('container');
2 createTree(container, data); // crée l'arbre dans le conteneur

```

Le résultat (arbre) devrait ressembler à ceci :

- Fish
 - trout
 - salmon
- Tree
 - Huge
 - sequoia
 - oak
 - Flowering
 - apple tree
 - magnolia

Choisissez l'une des deux façons de résoudre cette tâche :

1. Créez le code HTML de l'arborescence, puis attribuez-le à `container.innerHTML`.
2. Créez des nœuds d'arbre et ajoutez-les avec les méthodes DOM.

Ce serait génial si vous pouviez faire les deux.

P.S. L'arbre ne doit pas avoir d'éléments "supplémentaires" comme des `<ul></ul>` vides pour les feuilles (de l'arbre).

Nous allons créer dynamiquement un arbre HTML à partir d'un objet Javascript.. Chaque clé de l'objet deviendra un élément de liste, et si une clé contient un autre objet, nous créerons une sous-liste.

Nous commençons par définir un objet JavaScript qui représente une structure hiérarchique. Ensuite, nous créons une fonction createTree qui reçoit un conteneur HTML et un objet. Dans cette fonction, nous créons une liste , puis nous parcourons les clés de l'objet. Pour chaque clé, nous créons un élément et nous y ajoutons le texte correspondant. Si la valeur associée à la clé est un objet non vide, nous appelons récursivement la fonction afin de créer une sous-liste. Enfin, nous ajoutons la liste créée au conteneur. Cette méthode permet de construire l'arbre dynamiquement sans créer de listes vides.

```
<script>
let data = {
  "Fish": {
    "trout": {},
    "salmon": {}
  },
  "Tree": {
    "Huge": {
      "sequoia": {},
      "oak": {}
    },
    "Flowering": {
      "apple tree": {},
      "magnolia": {}
    }
  }
};

function createTree(container, obj) {
  let ul = document.createElement("ul");

  for (let key in obj) {
    let li = document.createElement("li");
    li.textContent = key;

    if (Object.keys(obj[key]).length > 0) {
      createTree(li, obj[key]);
    }

    ul.append(li);
  }

  container.append(ul);
}

let container = document.getElementById("container");
createTree(container, data);
</script>
```

- Fish
 - trout
 - salmon
- Tree
 - Huge
 - sequoia
 - oak
 - Flowering
 - apple tree
 - magnolia

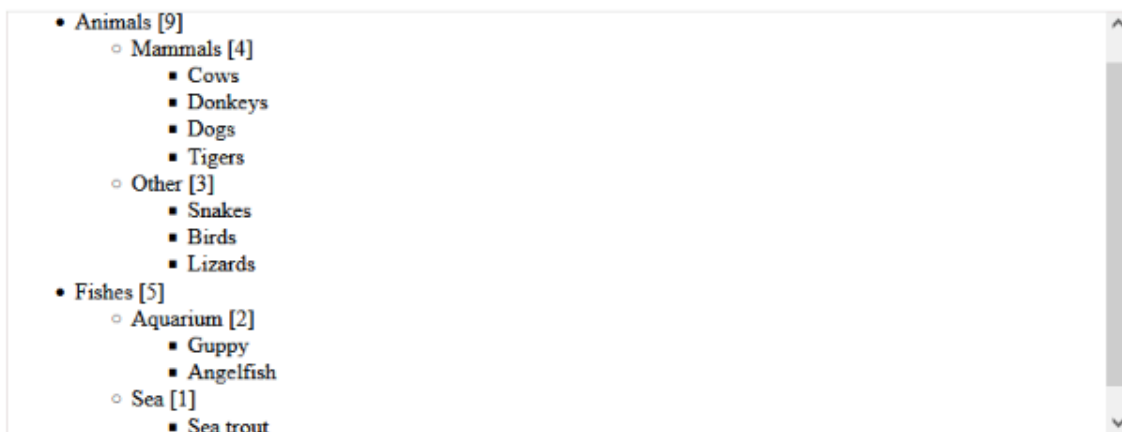
Afficher les descendants dans un arbre

Afficher les descendants dans un arbre

Il y a un arbre organisé comme un `ul/li` imbriqué.

Écrivez le code qui ajoute à chaque `<li>` le nombre de ses descendants. Sauter les feuilles (nœuds sans enfants).

Le resultat :



```
<script>
function countDescendants(li) {
  let descendants = li.querySelectorAll("li").length;

  if (descendants > 0) {
    li.firstChild.textContent += " [" + descendants + "]";
  }
}

let allLi = document.querySelectorAll("#tree li");

allLi.forEach(li => countDescendants(li));
</script>
```

- Animals [9]
 - Mammals [4]
 - Cows
 - Donkeys
 - Dogs
 - Tigers
 - Other [3]
 - Snakes
 - Birds
 - Lizards
- Fishes [5]
 - Aquarium [2]
 - Guppy
 - Angelfish
 - Sea [1]
 - Sea trout

Créer un calendrier

Créer un calendrier

Écrivez une fonction `createCalendar(elem, year, month)`.

L'appel doit créer un calendrier pour l'année/le mois donné et le mettre dans `elem`.

Le calendrier doit être un tableau, où une semaine est un `<tr>` et un jour est un `<td>`. Le dessus du tableau doit être un `<th>` avec les noms des jours de la semaine : le premier jour doit être le lundi, et ainsi de suite jusqu'au dimanche.

Par exemple, `createCalendar(cal, 2012, 9)` devrait générer dans l'élément `cal` le calendrier suivant :

MO	TU	WE	TH	FR	SA	SU
					1	2
3	4	5	6	7	8	9
10	11	12	13	14	15	16
17	18	19	20	21	22	23
24	25	26	27	28	29	30

P.S. Pour cette tâche, il suffit de générer le calendrier, il ne doit pas encore être cliquable.

Nous allons créer une fonction `createCalendar` qui génère dynamiquement un calendrier mensuel sous forme de tableau HTML. Chaque semaine sera une ligne (`<tr>`), chaque jour une cellule (`<td>`), et la première ligne contiendra les noms des jours de la semaine en commençant par lundi. Le calendrier sera ensuite affiché dans l'élément `elem`.

Nous commençons par créer un tableau HTML qui servira de calendrier. Nous ajoutons d'abord une ligne d'en-tête contenant les jours de la semaine, en commençant par lundi. Ensuite, nous utilisons l'objet Date pour déterminer le premier jour du mois et placer les cellules vides nécessaires avant le premier jour. Nous parcourons ensuite tous les jours du mois et créons une cellule <td> pour chaque date.

```

<script>
function createCalendar(elem, year, month) {
  let table = document.createElement("table");

  let days = ["MO", "TU", "WE", "TH", "FR", "SA", "SU"];
  let tr = document.createElement("tr");

  for (let day of days) {
    let th = document.createElement("th");
    th.textContent = day;
    tr.append(th);
  }
  table.append(tr);

  let date = new Date(year, month - 1, 1);
  let startDay = (date.getDay() + 6) % 7; // lundi = 0

  tr = document.createElement("tr");

  for (let i = 0; i < startDay; i++) {
    tr.append(document.createElement("td"));
  }

  while (date.getMonth() === month - 1) {
    let td = document.createElement("td");
    td.textContent = date.getDate();
    tr.append(td);

    if ((date.getDay() + 6) % 7 === 6) {
      table.append(tr);
      tr = document.createElement("tr");
    }

    date.setDate(date.getDate() + 1);
  }

  if (tr.children.length > 0) {
    table.append(tr);
  }

  elem.append(table);
}

```

MO	TU	WE	TH	FR	SA	SU
					1	2
3	4	5	6	7	8	9
10	11	12	13	14	15	16
17	18	19	20	21	22	23
24	25	26	27	28	29	30

Horloge colorée avec setInterval

Horloge colorée avec setInterval

Créez une horloge colorée comme ici :



Utilisez HTML/CSS pour le style, JavaScript ne met à jour que le temps dans les éléments.

Nous allons créer une horloge colorée qui affiche les heures, minutes et secondes. Nous allons utiliser HTML et CSS pour le style, puis JavaScript avec setInterval pour mettre à jour l'heure chaque seconde. Nous ajouterons également deux boutons Start et Stop pour démarrer et arrêter l'horloge.

```
<script>
  let timerId;

  function updateClock() {
    let now = new Date();

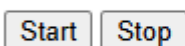
    let h = String(now.getHours()).padStart(2, "0");
    let m = String(now.getMinutes()).padStart(2, "0");
    let s = String(now.getSeconds()).padStart(2, "0");

    document.querySelector(".hours").textContent = h;
    document.querySelector(".minutes").textContent = m;
    document.querySelector(".seconds").textContent = s;
  }

  function startClock() {
    if (!timerId) {
      updateClock();
      timerId = setInterval(updateClock, 1000);
    }
  }

  function stopClock() {
    clearInterval(timerId);
    timerId = null;
  }
</script>
```

12 : 14 : 15



Trier le tableau

Trier le tableau

Il y a un tableau :

```
1 <table>
2 <thead>
3   <tr>
4     <th>Name</th><th>Surname</th><th>Age</th>
5   </tr>
6 </thead>
7 <tbody>
8   <tr>
9     <td>John</td><td>Smith</td><td>10</td>
10  </tr>
11  <tr>
12    <td>Pete</td><td>Brown</td><td>15</td>
13  </tr>
14  <tr>
15    <td>Ann</td><td>Lee</td><td>5</td>
16  </tr>
17  <tr>
18    <td>...</td><td>...</td><td>...</td>
19  </tr>
20 </tbody>
21 </table>
```

Il peut y avoir plus de lignes.

Écrivez le code pour le trier par la colonne "name".

Dans cet exercice, nous allons trier dynamiquement les lignes d'un tableau HTML en fonction de la colonne Name. Nous allons récupérer toutes les lignes du tableau, comparer les valeurs de la première colonne et réorganiser les lignes par ordre alphabétique à l'aide de Javascript.

```
<script>
  let tbody = document.querySelector("#table tbody");

  let rows = Array.from(tbody.rows);

  rows.sort((rowA, rowB) => {
    let nameA = rowA.cells[0].textContent;
    let nameB = rowB.cells[0].textContent;
    return nameA.localeCompare(nameB);
  });

  tbody.append(...rows);
</script>
```

Create a notification

Create a notification

Write a function `showNotification(options)` that creates a notification: `<div class="notification">` with the given content. The notification should automatically disappear after 1.5 seconds.

The options are:

```
1 // shows an element with the text "Hello" near the right-top of the window
2 showNotification({
3   top: 10, // 10px from the top of the window (by default 0px)
4   right: 10, // 10px from the right edge of the window (by default 0px)
5   html: "Hello!", // the HTML of notification
6   className: "welcome" // an additional class for the div (optional)
7 });
```

Notification is on the right

Hello 6

Lorem ipsum dolor sit amet, consectetur adipisicing elit. Dolorum aspernatur quam ex eaque inventore quod voluptatem adipisci omnis nemo nulla fugit iste numquam ducimus cumque minima porro ea quidem maxime necessitatibus beatae labore soluta voluptatum magnam consequatur sit laboriosam velit excepturi laborum sequi eos placeat et quia deleniti? Corrupti velit impedit autem et obcaecati fuga debitis nemo ratione iste veniam amet dicta hic ipsam unde cupiditate incidunt aut iure ipsum officiis soluta temporibus. Tempore dicta ullam delectus numquam consectetur quisquam explicabo culpa excepturi placeat quo sequi molestias reprehenderit hic at nemo cumque voluptates quidem repellendus maiores unde earum molestiae ad.

Use CSS positioning to show the element at given top/right coordinates. The source document has the necessary styles.

Nous allons créer une fonction `showNotification` qui affiche une notification en haut à droite de la fenêtre. Cette notification contiendra un message personnalisé, pourra avoir une classe CSS supplémentaire, sera positionnée grâce au CSS, puis disparaîtra automatiquement après 1,5 seconde.

```
<script>
function showNotification(options) {
  let div = document.createElement("div");
  div.className = "notification";

  if (options.className) {
    div.classList.add(options.className);
  }

  div.innerHTML = options.html;

  div.style.top = (options.top || 0) + "px";
  div.style.right = (options.right || 0) + "px";

  document.body.append(div);

  setTimeout(() => {
    div.remove();
  }, 1500);
}

showNotification({
  top: 10,
  right: 10,
  html: "Hello!",
  className: "welcome"
});
</script>
```

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Notification example.

Hello!

Nous avons créé une notification dynamique positionnée avec le CSS et affichée temporairement grâce à `setTimeout`.

Quel est le défilement

Quel est le défilement à partir du bas ?

La propriété `elem.scrollTop` est la taille de la partie déroulante à partir du haut. Comment obtenir la taille du défilement inférieur (appelons-le `scrollTopBottom`) ?

Écrivez le code qui fonctionne pour un `element` arbitraire.

P.S. Veuillez vérifier votre code: s'il n'y a pas de défilement ou que l'élément est entièrement défilé vers le bas, alors il devrait retourner `0`.

Nous allons calculer la distance de défilement restante depuis le bas d'un élément. Nous savons que `scrollTop` donne la distance défilée depuis le haut, donc nous allons utiliser les propriétés du DOM pour déterminer combien il reste à défiler jusqu'en bas.

```
<script>
  function getScrollBottom(elem) {
    return elem.scrollHeight - elem.scrollTop - elem.clientHeight;
  }

  function checkScroll() {
    let box = document.getElementById("box");
    alert(getScrollBottom(box));
  }
</script>
```

1
2
3
4
5
6

Vérifier scrollTopBottom

Cette page indique

200

OK

Quelle est la largeur de la barre de défilement ?

Quelle est la largeur de la barre de défilement ?

Écrivez le code qui renvoie la largeur d'une barre de défilement standard.

Pour Windows, il varie généralement entre 12px et 20px. Si le navigateur ne lui réserve pas d'espace (la barre de défilement est à moitié translucide sur le texte, cela arrive également), alors il peut s'agir de 0px.

P.S. Le code devrait fonctionner pour tout document HTML, ne dépend pas de son contenu.

Nous allons calculer la largeur de la barre de défilement du navigateur. Pour cela, nous allons comparer la largeur totale de la fenêtre avec la largeur réellement disponible pour le contenu. Le code devra fonctionner pour n'importe quel document HTML et retourner 0 si le navigateur n'affiche pas de barre de défilement visible.

```
<script>
  function getScrollbarWidth() {
    return window.innerWidth - document.documentElement.clientWidth;
  }

  function checkScrollbar() {
    alert(getScrollbarWidth() + " px");
  }
</script>
```

Nous avons utilisé `window.innerWidth`, qui correspond à la largeur totale de la fenêtre du navigateur, et `document.documentElement.clientWidth`, qui représente la largeur disponible pour le contenu sans la barre de défilement. La différence entre ces deux valeurs correspond à la largeur de la barre de

défilement. Si le navigateur n'affiche pas de barre de défilement visible, cette



différence est égale à 0.

Placer la balle au centre du terrain

Voici à quoi ressemble le document source :

```
index.html
1  <!DOCTYPE HTML>
2  <html>
3
4  <head>
5    <style>
6      #field {
7        width: 200px;
8        border: 10px groove black;
9        background-color: #00FF00;
10       position: relative;
11     }
12
13     #ball {
14       position: absolute;
15     }
16   </style>
17 </head>
18
19 <body>
20
21
22   <div id="field">
23      . . . . .
24     . . . . .
25     . . . . .
26     . . . . .
27     . . . . .
28   </div>
29
30 </body>
31 </html>
```

[ballon.html](#)

Quelles sont les coordonnées du centre de terrain ?

Calculez et utilisez-les pour placer la balle au centre du champ vert :

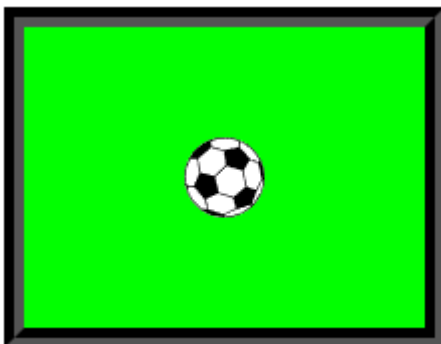
Dans cet exercice, nous allons calculer le centre du terrain vert et placer la balle exactement au centre en utilisant JavaScript uniquement. Nous allons récupérer les dimensions du terrain et de la balle, calculer les coordonnées du centre, puis déplacer la balle en modifiant ses propriétés left et top.

Nous commençons par récupérer le terrain et la balle à l'aide de `getElementById`. Ensuite, nous utilisons les dimensions visibles du terrain (`clientWidth` et `clientHeight`) ainsi que celles de la balle (`offsetWidth` et `offsetHeight`). Pour centrer la balle, nous calculons la position horizontale et verticale en prenant la moitié de la différence entre la taille du terrain et celle de la balle. Enfin, nous appliquons ces valeurs aux propriétés `left` et `top` de la balle, ce qui permet de la placer exactement au centre du terrain.

```
<script>
  let field = document.getElementById("field");
  let ball = document.getElementById("ball");

  ball.onload = function () {
    let left = (field.clientWidth - ball.offsetWidth) / 2;
    let top = (field.clientHeight - ball.offsetHeight) / 2;

    ball.style.left = left + "px";
    ball.style.top = top + "px";
  };
</script>
```



La différence : largeur CSS vs clientWidth

La différence: largeur CSS vs clientWidth

Quelle est la différence entre `getComputedStyle(elem).width` et `elem.clientWidth` ?

Donnez au moins 3 différences. Plus c'est mieux.

`GetComputedStyle(elem).width` retourne la largeur définie par le CSS sous forme de chaîne, tandis que `elem.clientWidth` retourne la largeur réelle utilisable par le contenu sous forme de nombre. `clientWidth` inclut le padding mais pas les bordures ni la barre de défilement, contrairement à la valeur CSS. Ces deux propriétés sont utilisées dans des contextes différents selon que l'on souhaite lire un style ou effectuer des calculs de mise en page.

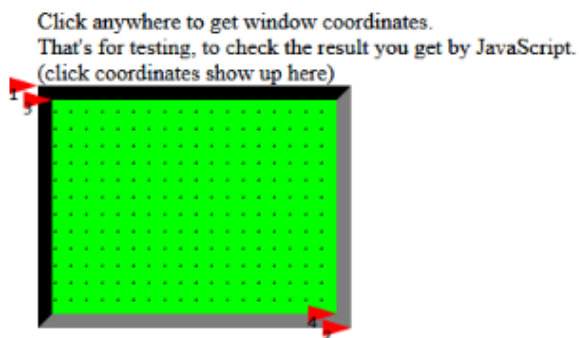
Trouver les coordonnées de la fenêtre du champ

Trouver les coordonnées de la fenêtre du champ

Dans l'iframe ci-dessous, vous pouvez voir un document avec le "champ" vert.

Utilisez JavaScript pour trouver les coordonnées de la fenêtre des coins pointés par des flèches.

Il y a une petite fonctionnalité implémentée dans le document pour plus de commodité. Un clic à n'importe quel endroit montre les coordonnées là-bas.



[champ.html](#)

Votre code doit utiliser DOM pour obtenir les coordonnées de la fenêtre de :

1. Coin extérieur supérieur gauche (c'est simple).
2. En bas à droite, coin extérieur (simple aussi).
3. Coin intérieur supérieur gauche (un peu plus dur).
4. En bas à droite, coin intérieur (il y a plusieurs façons, choisissez-en une).

Les coordonnées que vous calculez doivent être les mêmes que celles renvoyées par le clic de souris.

P.S. Le code devrait également fonctionner si l'élément a une autre taille ou bordure, qui n'est lié à aucune valeur fixe.

Dans cet exercice, nous allons utiliser JavaScript et le DOM pour trouver les coordonnées dans la fenêtre des coins du champ vert. Nous allons calculer :

le coin extérieur supérieur gauche
le coin extérieur inférieur droit
le coin intérieur supérieur gauche
le coin intérieur inférieur droit

```
<script>
  let field = document.getElementById("field");
  let coordsText = document.getElementById("coords");

  document.addEventListener("click", function (event) {
    coordsText.textContent =
      "x: " + event.clientX + ", y: " + event.clientY;
  });

  let rect = field.getBoundingClientRect();

  let outerTopLeft = {
    x: rect.left,
    y: rect.top
  };

  let outerBottomRight = {
    x: rect.right,
    y: rect.bottom
  };

  let innerTopLeft = {
    x: rect.left + field.clientLeft,
    y: rect.top + field.clientTop
  };

  let innerBottomRight = {
    x: innerTopLeft.x + field.clientWidth,
    y: innerTopLeft.y + field.clientHeight
  };

  console.log("Extérieur haut gauche:", outerTopLeft);
  console.log("Extérieur bas droite:", outerBottomRight);
  console.log("Intérieur haut gauche:", innerTopLeft);
  console.log("Intérieur bas droite:", innerBottomRight);
</script>
```

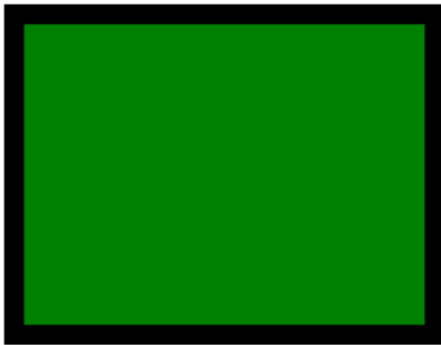
x: 61, y: 91



x: 248, y: 92



x: 68, y: 217



x: 246, y: 222



Afficher une note près de l'élément

Afficher une note près de l'élément

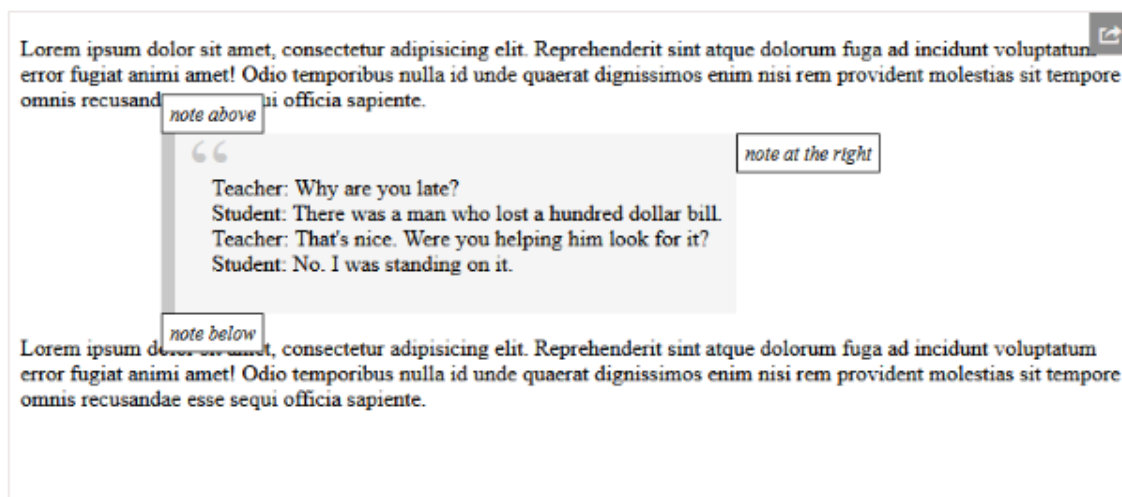
Créez une fonction `positionAt(anchor, position, elem)` qui positionne `elem`, en fonction de `position` près de l'élément `anchor`.

La `position` doit être une chaîne de caractères avec l'une des 3 valeurs :

- `"top"` – position `elem` juste au dessus de `anchor`
- `"right"` – position `elem` immédiatement à droite de `anchor`
- `"bottom"` – position `elem` juste en dessous `anchor`

Il est utilisé à l'intérieur de la fonction `showNote(anchor, position, html)`, fournie dans le code source de la tâche, qui crée un élément "note" avec `html` donné et l'affiche à la `position` donnée près de `anchor`.

Voici la démo des notes :



Nous allons créer une fonction `positionAt` qui permet de placer un élément par rapport à un autre élément appelé `anchor`. Selon la valeur de `position`, la note sera affichée au-dessus, à droite ou en dessous de l'élément cible. Tout le positionnement sera fait en JavaScript, à l'aide des coordonnées du DOM.

```

<script>
  function positionAt(anchor, position, elem) {
    let rect = anchor.getBoundingClientRect();

    if (position === "top") {
      elem.style.left = rect.left + "px";
      elem.style.top = rect.top - elem.offsetHeight + "px";
    }

    if (position === "right") {
      elem.style.left = rect.right + "px";
      elem.style.top = rect.top + "px";
    }

    if (position === "bottom") {
      elem.style.left = rect.left + "px";
      elem.style.top = rect.bottom + "px";
    }
  }

  function showNote(anchor, position, html) {
    let note = document.createElement("div");
    note.className = "note";
    note.innerHTML = html;

    document.body.append(note);
    positionAt(anchor, position, note);
  }

  let anchor = document.getElementById("anchor");

  showNote(anchor, "top", "note above");
  showNote(anchor, "right", "note at the right");
  showNote(anchor, "bottom", "note below");
</script>

```

nous avons positionné dynamiquement une note par rapport à un élément cible en utilisant les coordonnées fournies par le DOM.

